

```
# This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory
```

✓ Stable diffusion 3.5

Будем использовать модель Stable diffusion 3.5 medium. Если у вас нет регистрации hugging face, зарегистрируйтесь. Затем запросите доступ к модели. Подгрузим библиотеки, определим девайс и создадим диффузионный пайплайн. В библиотеке diffusers есть пайплайны для использования уже готовых моделей, в том числе для Stable diffusion 3.5. Подробно разберём его в следующем уроке, а сейчас просто используем. В своём первоизданном виде этот пайплайн занимает много памяти, поэтому используем квантизованные версии моделей из пайплайна из библиотеки bitsandbytes. Установим библиотеку: `pip install -U bitsandbytes`

```
# Установка нужной версии
!pip install -U -q bitsandbytes>=0.46.1

# Перезапуск ядра (Kernel)
import os
os._exit(0)
```

```
#pip install -U bitsandbytes
```

```
from diffusers import BitsAndBytesConfig, SD3Transformer2DModel
from diffusers import StableDiffusion3Pipeline, FlowMatchEulerDiscreteScheduler
from transformers import T5EncoderModel
from transformers import BitsAndBytesConfig as TransformersBitsAndBytesConfig
import torch
import matplotlib.pyplot as plt

device = "cuda" if torch.cuda.is_available() else "cpu"
```

```
if torch.cuda.is_available():
    print(f"Доступна CUDA! Устройство: {torch.cuda.get_device_name(0)}")
    device = torch.device("cuda")
```

```
# модель которую будем использовать
model_id = "stabilityai/stable-diffusion-3.5-medium"
```

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
hf_token = user_secrets.get_secret("HF_TOKEN")

# Теперь переменную hf_token можно передавать в .from_pretrained(...)
```

```
import torch
import gc
from diffusers import StableDiffusion3Pipeline
from transformers import T5EncoderModel, BitsAndBytesConfig
from kaggle_secrets import UserSecretsClient

# 1. Получение токена из секретов Kaggle
user_secrets = UserSecretsClient()
hf_token = user_secrets.get_secret("HF_TOKEN")

model_id = "stabilityai/stable-diffusion-3.5-medium"

# 2. Очистка кэша перед загрузкой
gc.collect()
torch.cuda.empty_cache()

# 3. Конфигурация квантования для экономии VRAM
```

```

quant_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16
)

# 4. Загрузка тяжелого текстового энкодера (T5) в 4 битах
text_encoder = T5EncoderModel.from_pretrained(
    model_id,
    subfolder="text_encoder_3",
    quantization_config=quant_config,
    torch_dtype=torch.float16,
    token=hf_token,
    device_map="auto" # Для отдельного компонента auto работает хорошо
)

# 5. Загрузка основного пайплайна
# Мы не используем device_map здесь, чтобы применить cpu_offload позже
pipe = StableDiffusion3Pipeline.from_pretrained(
    model_id,
    text_encoder_3=text_encoder,
    torch_dtype=torch.float16,
    token=hf_token
)

# 6. КРИТИЧЕСКИЕ ОПТИМИЗАЦИИ ДЛЯ KAGGLE
# Переносит части модели на CPU, когда они не активны
pipe.enable_model_cpu_offload()
# Включает нарезку внимания для экономии памяти при больших разрешениях
pipe.enable_attention_slicing()

# 7. Генерация
# Если всё равно будет OOM, попробуйте уменьшить height и width до 768
image = pipe(
    prompt="a photo of a cat holding a sign that says hello world",
    negative_prompt="",
    num_inference_steps=28,
    height=1024,
    width=1024,
    guidance_scale=7.0,
).images[0]

# 8. Сохранение результата
image.save("result.png")
print("Изображение успешно сохранено как result.png")

```

Напишите программный код или сгенерируйте его с помощью искусственного интеллекта.